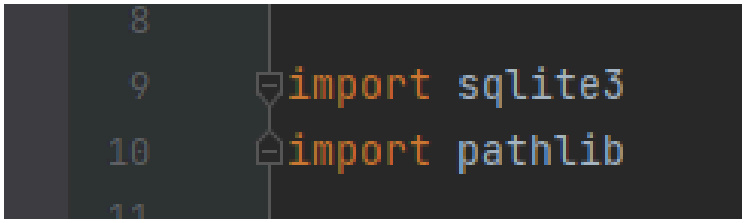
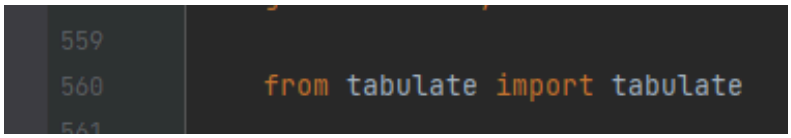
Criterion C: Development

List of techniques used:

- Additional libraries
- Database creation
- Table creation
- Inputted data into a database
- Data extraction from the database
- Mean calculations
- Use of 2D arrays
- Continue statement
- Error checking
- For loops

Use of additional libraries:







The “import” function is used to import external libraries into the PyCharm IDE to be used with the python code.

The “sqlite3” library was used to easily create and manage databases.

“Pathlib” was used to detect if the file already exists and if it was the first run.

“Tabulate” was used to create nicely formatted tables.

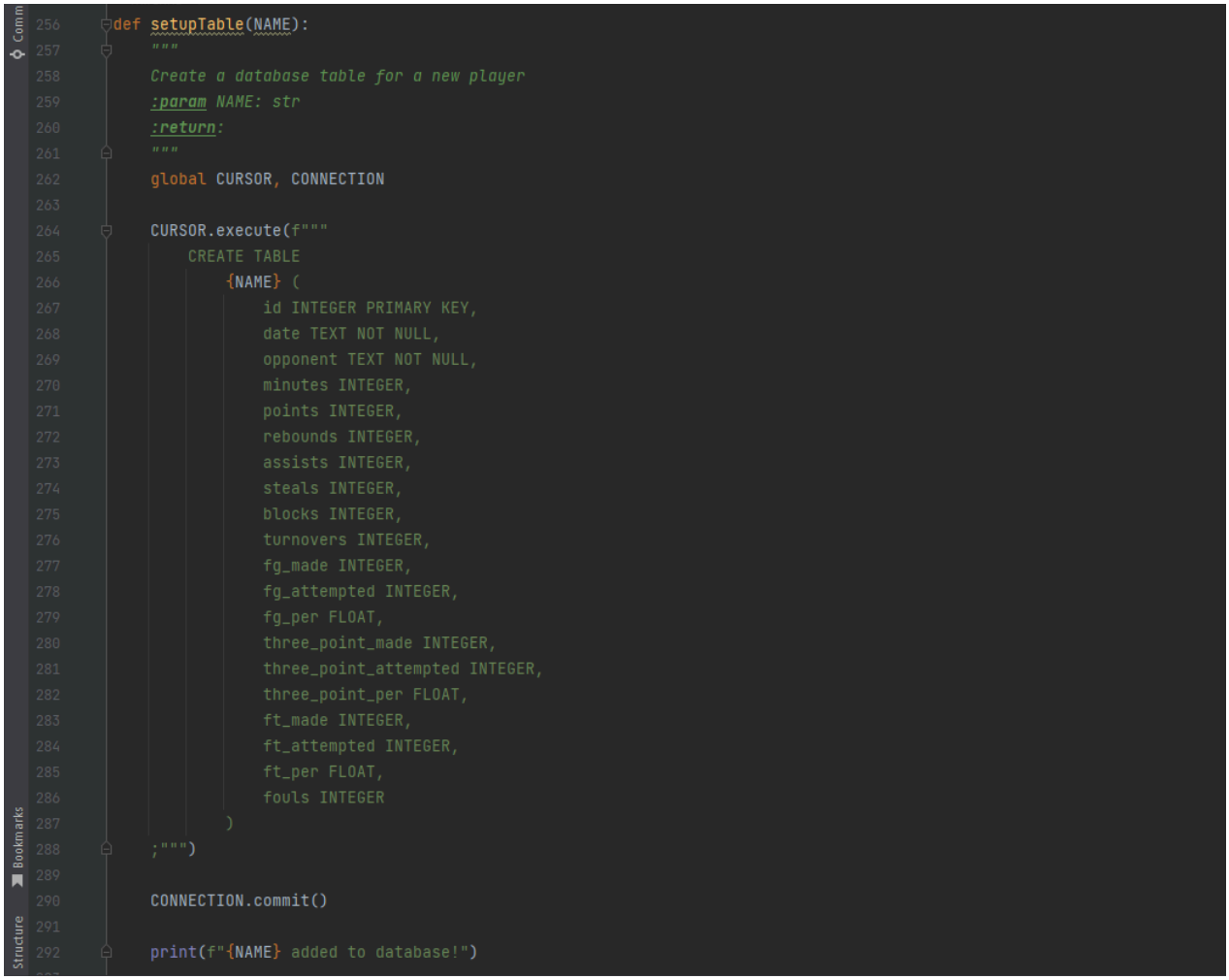
“Numpy” was used to easily make calculations such as the mean.

Database creation:

```
13  FILENAME = "stats.db"
14
15  # TEST if the FILENAME already exists
16
17  FIRST_RUN = True
18
19  if (pathlib.Path.cwd() / FILENAME).exists():
20      FIRST_RUN = False
21
22  CONNECTION = sqlite3.connect(FILENAME)
23  CURSOR = CONNECTION.cursor()
24
```

There is an error check used initially when the program is run to test if the database already exists. One database was created to store multiple tables for players and a singular team table. On the first run, with the variable “FIRST_RUN” being true, a database is created titled “stats.db.” Using the “pathlib” library, the program will detect if the database already exists and will not create a new database due to the variable “FIRST_RUN” being false.

Table creation:



```
def setupTable(NAME):
    """
    Create a database table for a new player
    :param NAME: str
    :return:
    """
    global CURSOR, CONNECTION

    CURSOR.execute(f"""
        CREATE TABLE
            {NAME} (
                id INTEGER PRIMARY KEY,
                date TEXT NOT NULL,
                opponent TEXT NOT NULL,
                minutes INTEGER,
                points INTEGER,
                rebounds INTEGER,
                assists INTEGER,
                steals INTEGER,
                blocks INTEGER,
                turnovers INTEGER,
                fg_made INTEGER,
                fg_attempted INTEGER,
                fg_per FLOAT,
                three_point_made INTEGER,
                three_point_attempted INTEGER,
                three_point_per FLOAT,
                ft_made INTEGER,
                ft_attempted INTEGER,
                ft_per FLOAT,
                fouls INTEGER
            )
    ;""")

    CONNECTION.commit()

    print(f"{NAME} added to database!")
```

The variable “NAME” is used to create an SQL table for an individual player inputted by the user. The table is created with an ID integer to identify the row. It is possible to have multiple rows within the table. Within the SQL command there are columns within the table representing statistical categories for the individual player. “Fg” represents field goals, “three point” represents three point field goals and “ft” represents free throws.

```
434  def setupTeamTable():
435      """
436      Set up a table for the team stats
437      :return:
438      """
439
440      global CURSOR, CONNECTION
441
442      CURSOR.execute("""
443          CREATE TABLE
444              Lillian_Osborne_Basketball (
445                  id INTEGER PRIMARY KEY,
446                  name TEXT NOT NULL,
447                  avg_minutes INTEGER,
448                  avg_points INTEGER,
449                  avg_rebounds INTEGER,
450                  avg_assists INTEGER,
451                  avg_steals INTEGER,
452                  avg_blocks INTEGER,
453                  avg_turnovers INTEGER,
454                  avg_fg_made INTEGER,
455                  avg_fg_attempted INTEGER,
456                  avg_fg_per FLOAT,
457                  avg_three_point_made INTEGER,
458                  avg_three_point_attempted INTEGER,
459                  avg_three_point_per FLOAT,
460                  avg_ft_made INTEGER,
461                  avg_ft_attempted INTEGER,
462                  avg_ft_per FLOAT,
463                  avg_fouls INTEGER
464                  )
465      ;""")
466
467      CONNECTION.commit()
```

Using the inputted player name, the program creates a table within the database using the player name as the name of the table. Multiple player tables are able to be created and added into the database. The program also creates a singular team table on the first run. The table is titled “Lillian_Osborne_Basketball” and has values inside the table representing the average of a certain statistical category for an individual player, and can have multiple rows representing multiple players. The name, date, and opponent values cannot be null while the other rows can be left as blank or 0.

There are values in the table set as integers or floats meaning that the value has to be set as an integer to be added to the table.

Inputted data into a database:

```python
def getPlayerName():
    """
    User inputs the name of the player
    :return: str
    """
    FIRST_NAME = input("Enter first name: ")
    LAST_NAME = input("Enter last name: ")
    PLAYER = FIRST_NAME + "_" + LAST_NAME
    PLAYERS.append(PLAYER)
    return PLAYER

```

```python
    CURSOR.execute(f"""
        INSERT INTO
            {NAME} (
                date,
                opponent,
                minutes,
                points,
                rebounds,
                assists,
                steals,
                blocks,
                turnovers,
                fg_made,
                fg_attempted,
                fg_per,
                three_point_made,
                three_point_attempted,
                three_point_per,
                ft_made,
                ft_attempted,
                ft_per,
                fouls
            )
        VALUES (
            ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
        )
    ;""", NEW_GAME)

    CONNECTION.commit()

```

The user inputs information for multiple functions of the program. The user inputs the name of the player as well as the values for the player stats. The user enters the name of the player when they choose to add a new player. The first and last name is joined using an underscore to be used as a singular string. The name string is used to create a table for the individual player and identify the player to allow the user to interact with the table. When the user chooses to edit the player, they can choose to add a new row, where they are prompted to enter a value for every column, representing each statistical category. The program has some error checking to filter out bad values entered. For the percentage value for “fg_per,” “three_point_per” and “ft_per,” The percentage is calculated by dividing the shots made(fg_made, three_point_made, ft_made) by the attempts(fg_attempted, three_point_attempted, ft_attempted).

Data extraction from the database:

```python
def displayPlayer(NAME):
    """
    Display the table for a player
    :param NAME: str
    :return:
    """

    global CURSOR, CONNECTION

    from tabulate import tabulate

    STATS = CURSOR.execute(f"""
            SELECT
                *
            FROM
                {NAME}
        """).fetchall()

    TABLE = STATS
    HEADER = ['DATE', 'OPP', 'MIN', 'PTS', 'REB', 'AST', 'STL', 'BLK', 'TO', 'FGM', 'FGA', 'FG%', '3PM', '3PA', '3P%',
              'FTM', 'FT_A', 'FT%', 'F']

    print(tabulate(TABLE, headers=HEADER))
```

```python
def displayTeam():
    """
    Display the table for a player
    :param NAME: str
    :return:
    """

    global CURSOR, CONNECTION

    from tabulate import tabulate

    STATS = CURSOR.execute(f"""
            SELECT
                *
            FROM
                Lillian_Osborne_Basketball
        """).fetchall()

    TABLE = STATS
    HEADER = ['NAME', 'MPG', 'PPG', 'RPG', 'APG', 'SPG', 'BPG', 'TPG', 'FGM', 'FGA', 'FG%', '3PM', '3PA', '3P%',
              'FTM', 'FT_A', 'FT%', 'FPG']

    print(tabulate(TABLE, headers=HEADER))
```

The program extracts the data from each of the tables to be viewed in a formatted table by the viewer. The tables being selected are the individual player tables and the singular team table. The select * is used to select everything from the table. .fetchall() ensures that every value is selected. Tabulate is used to format all of these values into a table that is printed out and labeled headers are used for every column. Tabulate uses the variable “TABLE” to input all of the data which is the data extracted from the table and adds in a header which is customized titles for each column.

Mean calculations:

```
328     import numpy as np
329
330     STAT = np.array(STATS)
331
332     MIN_AVG = STAT[1:, 0].mean()
333     PTS_AVG = STAT[1:, 1].mean()
334     REB_AVG = STAT[1:, 2].mean()
335     AST_AVG = STAT[1:, 3].mean()
336     STL_AVG = STAT[1:, 4].mean()
337     BLK_AVG = STAT[1:, 5].mean()
338     TO_AVG = STAT[1:, 6].mean()
339     FGM_AVG = STAT[1:, 7].mean()
340     FGA_AVG = STAT[1:, 8].mean()
341     FGP_AVG = STAT[1:, 9].mean()
342     TPM_AVG = STAT[1:, 10].mean()
343     TPA_AVG = STAT[1:, 11].mean()
344     TPP_AVG = STAT[1:, 12].mean()
345     FTM_AVG = STAT[1:, 13].mean()
346     FTA_AVG = STAT[1:, 14].mean()
347     FTP_AVG = STAT[1:, 15].mean()
348     F_AVG = STAT[1:, 16].mean()
349
350     AVG = [MIN_AVG, PTS_AVG, REB_AVG, AST_AVG, STL_AVG, BLK_AVG, TO_AVG, FGM_AVG, FGA_AVG, FGP_AVG, TPM_AVG, TPA_AVG,
351            TPP_AVG, FTM_AVG, FTA_AVG, FTP_AVG, F_AVG]
352
353     return AVG
```

The numpy library is used to calculate the mean of each value. A variable is used for the mean of each column in one table in the database, then sent to the team table where the player's stat average is displayed. Each mean value is added to an array where it can be used to input values into a single row within a table.

Use of 2D arrays:

When the data is being extracted a 2D array is being used. During the table formatting the data is extracted as a 2D array and the data is displayed as a table. The data is also extracted as a 2D array to calculate the mean of the values.

Continue statement:

```
        elif CHOICE == 4:
            print("""
1. Add new game
2. Delete row
3. Delete player
4. Exit
            """)
            SUBMENU = int(input("> "))
            if SUBMENU == 1:
                NAME = getPlayerID()
                addNewRow(NAME)
                AVG = getPlayerAverage(NAME)
                updateAverage(NAME, AVG)
                updateTeamAverage(NAME, AVG)
            elif SUBMENU == 2:
                NAME = getPlayerID()
                ROW = getRowID(NAME)
                deleteRow(NAME, ROW)
            elif SUBMENU == 3:
                NAME = getPlayerID()
                deleteTable(NAME)
            else:
                continue
        else:
            exit()
```

The program uses the continue function to go back to the main menu. If the user selects the submenu, then exits the submenu, the continue function causes the program to exit the loop for the submenu and return to the main menu loop, without ending the program entirely.

Error checking:

```
    FG_MADE = input("Field goals made> ")
    if FG_MADE == "":
        FG_MADE = 0
    elif FG_MADE.isnumeric() is False:
        FG_MADE = input("Please enter a number> ")
    FG_MADE = int(FG_MADE)
    FG_ATTEMPTED = input("Field goals attempted> ")
    if FG_ATTEMPTED == 0 or "":
        FG_ATTEMPTED = 0
    elif FG_ATTEMPTED.isnumeric() is False:
        FG_ATTEMPTED = input("Please enter a number> ")
    FG_ATTEMPTED = int(FG_ATTEMPTED)
    if FG_ATTEMPTED > 0:
        FG_PER = (FG_MADE / FG_ATTEMPTED) * 100
        round(FG_PER, 1)
    elif FG_ATTEMPTED == 0:
        FG_PER = 0
```

```
    if ROW_INDEX == "" or 0:
        ROW_INDEX = input("Please choose a valid option> ")
    elif ROW_INDEX.isnumeric() is False:
        ROW_INDEX = input("Please choose a valid option> ")
    elif int(ROW_INDEX) > len(ALLPLAYERS):
        ROW_INDEX = input("Please choose a valid option> ")
```

The program has some forms of error checking. If the user enters a blank string into the add row option, the program will input 0 into the database. If the user enters a non-number character, the program will redirect the user and warn them to enter a number, or else the number that the user enters will be typecasted into an integer so that the calculations are able to be done in python. The percentage of the field goals, three point field goals and free throws are only calculated if there is a number of attempts, as a division by 0 will cause an error. The percentage is calculated by dividing the amount made by the amount of attempts and multiplying by 100.

If the user enters an invalid row index(there are numbered options given to the player to select), the program warns the user to enter a valid option.

For loops:

When the program adds the inputted information, a for loop is used to add all of the individual indexes in the list.

```
def getPlayerID():
    """
    User enters the id to search for a player
    :return: str
    """

    global CURSOR

    PLAYERS = CURSOR.execute("""
        SELECT
            name
        FROM
            sqlite_schema
        WHERE
            type = 'table'
    ;""").fetchall()

    ALLPLAYERS = list(sum(PLAYERS[1:], ()))

    for i in range(len(ALLPLAYERS)):
        print(f"{i + 1}: {ALLPLAYERS[i]}")

    ROW_INDEX = input("> ")
```

The program also uses a for loop to get the id of the row for the user to select a row within a table. When the user enters the number of the row they want to select, all of the values in the

row are printed. The program uses the name of the table in the “fetchall” command to get the amount of tables for individual players. The for loop is used to print the number of players for how many tables there are.

Word count: 1092